

...to witness the onboard LED light up for a few seconds before it turns off. The *pyboard* is a nice tool to experiment with running a few MicroPython one-liners on your RPI Pico boards in a non-interactive mode. It can also run MicroPython code from source file(s). Let's run *tglobled.py* through the following command:

```
docker run -it --rm --device=/dev/ttyACM0:/dev/ttyACM0 -v
${PWD}/tglobled.py:/etc/rpipico/tglobled.py:ro pyboard
tglobled.py
```

You can also upgrade the RPI Pico board MicroPython firmware through *pyboard* and the local command line, without pressing the BOOTSEL button and plugging the micro-USB physical actions. To upgrade the MicroPython firmware to the latest version, execute the command:

```
wget -r --tries=10 "https://micropython.org/${curl -s
https://micropython.org/download/RPI_PICO/|grep '\.uf2'|head
-1|grep RPI|awk -F '""' '{print $2}')" -o log -O /tmp/RPI_
PICO-Latest.uf2
```

...to download the latest firmware for your RPI Pico board. Now execute the command:

```
docker run --rm --device=/dev/ttyACM0:/dev/ttyACM0 pyboard -c
'import machine;machine.bootloader()'
```

...to put your board in the bootloader mode mounted as a local directory on your computer. The next command is:

```
cp /tmp/RPI_PICO-Latest.uf2 $(mount|grep RPI|awk '{print
$3}')
```

This copies the latest downloaded firmware to the board. Once the new firmware is programmed, the device will automatically reset and be ready for use. This way you can fully automate the RPI Pico board firmware upgrade and quick testing. Isn't it amazing?

Let's now explore the next command line tool *mpremote*, which has way more features than *pyboard*. Execute the following command:

```
docker build . -f Dockerfile_Mpremove -t mpremove
```

...to create the Docker image:

```
--- Start of Dockerfile_Mpremove ---
```

```
FROM python:3.14-alpine
```

```
SHELL ["/bin/ash", "-o", "pipefail", "-c"]
```

```
WORKDIR /tmp
RUN wget https://github.com/micropython/micropython/archive/
refs/heads/master.zip \
  && unzip master.zip \
  && cp -r micropython-master/tools/mpremote /usr/local/bin/
\
  && rm -rf master.zip micropython-master \
  && pip install -r /usr/local/bin/mpremote/requirements.txt
\
  && ln -s /usr/local/bin/mpremote/mpremote.py /usr/local/
bin/mpr \
  && mkdir /etc/rpipico
WORKDIR /etc/rpipico
ENTRYPOINT ["mpr"]
CMD ["--help"]
```

```
--- End of Dockerfile_Mpremove ---
```

Now execute the command:

```
docker run --rm --device=/dev/ttyACM0:/dev/ttyACM0 mpremove
```

This brings up a help screen to show all that you can do with it. You can use the *exec* command to run any code contained in a passed string like we did through the *pyboard*.

Use the *run* command to execute a local MicroPython code. You can execute the command:

```
docker run -it --rm --device=/dev/ttyACM0:/dev/ttyACM0
mpremote repl
```

...to launch an interactive shell running the MicroPython board. This provides many shortcuts to accomplish several common tasks for your board. You can also interact with multiple boards plugged into your desktop:

```
docker run -it --rm --device=/dev/ttyACM0:/dev/ttyACM0
--device=/dev/ttyACM0:/dev/ttyACM1 mpremove devs
```

...to list the available serial ports. In fact, you can do a lot using it on the command line. Do explore the *mpremote* through its official documentation link provided in the Reference section. Going forward, with *pyboard* and *mpremote*, there will be no further dependency on an IDE to play with and program your Pico boards.

Real-world applications of PI Pico

Let's start with the traffic light module shown in Figure 2. Figure 3 shows the wired circuit of the external LED traffic lights. The logic of the code is simple: configure 3 GPIO pins (choose from GPIO pin numbers in the board diagram) for